

20-Minute Supervisor Meeting Script

Project: Platform for Protecting Water Chlorination Systems from Cyber Attacks **Domain:** OT / ICS Security **Status:** Local end-to-end MVP complete (Phase 5A) · 286 tests passing **Purpose:** Spoken walkthrough — where we are, what we built, what we proved, what's next.

Delivery note: this is a script to say, not to read verbatim. Talk to it. Timings are targets, not rules. Pause points are marked [PAUSE — “Does that make sense?”].

0. Opening / Project Goal — ~1.5 min

Say:

“Thanks for the time. Let me walk you through where the project is today — what we built, why we built it this way, what we've actually proven, and what comes next before final delivery.

The project is a *Platform for Protecting Water Chlorination Systems from Cyber Attacks*. It sits in OT/ICS security — operational technology, the industrial control systems that run physical processes like water treatment.

The problem is this: water-treatment plants run on PLCs, sensors, pumps, and valves that were designed for reliability, not for attackers on the network. If someone manipulates a chlorination process — say, forces a dosing pump or spoofs a sensor reading — it can be a safety-critical event. And the tricky part is that from a traditional IT-security point of view, that malicious command can look like perfectly valid control traffic.

So the question we set out to answer is: *can we detect an attack from the physical behaviour of the plant itself* — from the process telemetry — rather than from network signatures? And then, can we wrap that detection in a real, engineered monitoring system that an operator could actually use?”

- **Show:** the README title section / one-line project statement.
 - **Key point:** this is a *process-aware* security problem, not a packet-inspection problem.
-

1. What We Built — ~2 min

Say:

“At a high level, what we built is a pipeline that takes water-treatment telemetry, runs it through an anomaly-detection model, turns detections into alerts, and shows those alerts to an operator on a dashboard.

The flow is: SWaT telemetry → anomaly detection with a model called TranAD → an alert engine → a monitoring dashboard.

The important framing for today is: this is *not just an ML model*. A lot of student projects stop at ‘we trained a model and got a number.’ We deliberately treated this as a *systems engineering* problem: we took a validated detection model and productionized it into an end-to-end, locally-running security-monitoring platform — with a replay layer, a real HTTP API, an alerting layer with proper lifecycle, a dashboard, and a full automated test suite.

Everything I’m about to describe runs today, locally, end-to-end. The MVP is complete.”

- **Show:** the architecture diagram in the README (the Mermaid flowchart).
- **Key point:** engineered platform, not a notebook. It runs end-to-end today.

[PAUSE — “Does that make sense so far?”]

2. The ML Model — TranAD — ~3 min

Say:

“Let me talk about the detection model, because that’s the heart of it.

During the research phase we investigated multiple anomaly-detection models and compared them on the SWaT dataset. We selected **TranAD** — a transformer-based reconstruction model. That selection is *closed*; we’re not re-opening it for the MVP. TranAD is the actual model the MVP uses, and its trained artifacts are frozen and treated as immutable.

Conceptually, TranAD is a reconstruction model. It learns what *normal* plant behaviour looks like, and for each slice of telemetry it tries to reconstruct it. When the plant is behaving normally, it reconstructs well — low error. When something is off — an attack, an abnormal state — it *can’t* reconstruct it well, and the reconstruction error spikes. That error is our anomaly score.

A few concrete numbers that define the model’s contract:

- It looks at **45 process variables** — sensors and actuators across the treatment stages.
- It works on a **window of 30 samples** at a time, not a single reading.
- The data is sampled at an **effective 5 seconds per sample** — the original source is 1 Hz, and the research pipeline kept every fifth row.
- So 30 samples × 5 seconds gives us **150 seconds of context** per decision. The model is always reasoning about the last two and a half minutes of plant behaviour, not an instantaneous snapshot.

The preprocessing has to match exactly what the model was trained on — standard-scaling per feature, a specific float32 cast, a flatten-and-min-max step, reshaped back to 30×45. We reproduced that chain bit-for-bit and we have regression tests locking it in, because if you get the preprocessing subtly wrong, the model still runs — it just silently produces meaningless scores.

Then there’s a **threshold**: if the anomaly score is above it, we flag the window as anomalous.

And one feature I want to highlight because it’s the ‘process-aware’ part: because TranAD reconstructs each sensor, we don’t just get a single score — we get a **per-feature error breakdown**. So when we

raise an alert we can say *which sensors drove it*. That's the difference between 'something is wrong' and 'these specific sensors are behaving abnormally.'"

- **Show:** README §4 (model contract table + preprocessing chain), and `ml/configs/tranad_swat.yaml` if he wants the exact contract.
- **Key point:** reconstruction error = anomaly score; 150 s of context; per-feature attribution is the explainability story.
- **Don't over-explain:** the transformer internals / attention math. Stay at "learns normal, flags what it can't reconstruct."

[PAUSE — "Does that make sense?"]

3. SWaT Dataset & Data Challenges — ~2 min

Say:

"The dataset is **SWaT — Secure Water Treatment** — from iTrust in Singapore. It's a real, physical water-treatment testbed with labelled normal and attack periods. It's the closest public analog we have to our chlorination use case. It's licensed, so it lives locally and is *not* committed to our repo — we point the code at it through an environment variable.

Now, this is where the project became genuinely an engineering-and-research effort rather than 'download data, train, done.' When we investigated the data provenance, we found real problems with the naive dataset:

- First, the commonly-used merged file has a **duplicated normal-data issue** — nearly a million rows are the same normal run present twice. That artificially inflates the amount of normal data and deflates the apparent attack ratio.
- Second, there was a **column header mismatch** that, combined with a blanket forward-fill in the original code, left **six of the model's features frozen as constants** across the validation split.
- And that directly **contaminated the threshold calibration** — the threshold was computed on data where six features were effectively dead.

So instead of blindly trusting `merged.csv`, we excluded it, and we use a **reconstructed clean series we call `Attack_v0`** — the attack rows plus the correct normal rows, merged by timestamp to restore the true normal → attack → normal ordering, with no duplicates.

We documented this entire investigation in a provenance record. The honest consequence is that our threshold has a known calibration caveat, which I'll come back to when we talk about results. The point I want to land here: we treated the data critically. We found the defects, documented them, and made a deliberate, transparent decision about how to handle them."

- **Show:** `docs/provenance/tranad_swat_provenance.md` and `docs/provenance/measured_detection_metrics.md` (the "ladder" table is powerful here).
- **Key point:** we didn't trust the dataset blindly — we found and documented real defects; that's a research contribution.
- **Don't over-explain:** the row-range specifics unless asked. "Duplicated normal run + a header bug that froze six features" is enough.

[PAUSE — "Does that make sense?"]

4. Productionization / Architecture — ~3 min

Say:

“Now the engineering. Here’s the actual architecture, layer by layer:

SWaTReplay loads that clean Attack_v0 series and streams it out as an ordered sequence of telemetry records — it simulates a live feed.

RollingWindow is the component that *owns the 30-sample buffer*. You push one telemetry record at a time; it holds the last 30 and emits a complete 30×45 window with stride 1. This is the *only* place in the system that knows about ‘the last 30 samples.’

That window then goes over HTTP to the **FastAPI backend**, which is deliberately a *thin, stateless* boundary. It validates the shape and the feature names, hands the window to the detector, and shapes the response. It keeps no per-request state.

The **TranADDetector** is **stateless** too. You give it a 30×45 window, it runs the preprocessing and the model, and it returns a score, the threshold decision, and the per-feature errors. It holds no buffering, no history.

Then the **AlertEngine** — this is the *one intentionally stateful* stage. It takes each per-window decision and manages alert *lifecycle*: it opens an alert on the first anomalous window, deduplicates the continuous run of anomalies into a single incident, and closes it when the plant returns to normal.

On top of that sits the **monitoring API** — read-only endpoints for status, alert history, and the active alert — and a **dashboard** that consumes *only* the API and polls it.

Now, *why* did we draw the boundaries this way? Because state is where distributed systems get hard. By pushing the buffer out to the caller and keeping the detector and the API stateless, the scoring layer becomes trivially testable and could scale horizontally later without shared state. We isolated all the mutable state into exactly one component — the AlertEngine — so we always know where ‘the system’s memory’ lives. That’s a deliberate design decision, and it’s what lets the different parts of the project be worked on and tested independently.

One honest note: the alert state is currently **in-memory only** — if the backend restarts, the alert history is gone. That’s by design for the MVP, and adding persistence is the very next step. I’ll come back to that.”

- **Show:** README §3 architecture table (component / state columns) and the Mermaid diagram; optionally backend/app.py to show how thin it is.
- **Key point:** state is isolated on purpose — buffer in RollingWindow, everything stateless except the AlertEngine, which is in-memory for now.
- **Don’t over-explain:** the CORS config, the exact Pydantic validation rules — mention they exist, move on.

[PAUSE — “Does that make sense?”]

5. What We Have Actually Proven — ~2 min

Say:

“Let me be precise about what’s *verified*, not just claimed.

We have **286 automated tests passing**. They cover every layer: the ML preprocessing chain, model loading, the scoring math, artifact integrity, the replay, the windowing, the API, the alert lifecycle, the monitoring endpoints, and full end-to-end integration.

Two things I want to emphasize. First, we have **golden-vector tests** — we froze known-good inputs and their exact expected scores, so if anything ever changes the numerical behaviour of the model or preprocessing, a test fails immediately. That’s how we guarantee we’re faithfully reproducing the research pipeline.

Second, the end-to-end test isn’t a mock. We ran the whole pipeline against **real SWaT data through a real HTTP server** — a real Uvicorn socket, not just an in-process function call. So we’ve proven the pipeline works over the actual network boundary, with real serialization and validation, on real data. On clean segments it stays normal; on the attack segment it flags the attack.”

- **Show:** run pytest live if time allows (286 passing), or the test directory listing; tests/test_golden_vectors.py and tests/test_integration_e2e.py as highlights.
 - **Key point:** 286 tests, golden-vector numerical lock, and a *real HTTP* end-to-end run on *real* data.
-

6. Live MVP Demonstration Story — ~2.5 min

Say:

“Here’s the demo story — the narrative we’d show live.

Normal: we replay a clean SWaT segment. Each window gets scored, the score stays *below* the threshold, and the dashboard shows NORMAL. No alerts.

Attack: we replay a real SWaT *attack* segment. The reconstruction error climbs, the score crosses the threshold, and the window is flagged as an anomaly. The AlertEngine **opens one alert** — and here’s the lifecycle detail: the next anomalous windows don’t spam new alerts, they get **deduplicated** into that same incident. The dashboard shows the active alert with its **top contributing sensors** — the process-aware evidence for *why* it fired.

Recovery: telemetry returns to normal, the score drops back below threshold, and the alert **closes automatically** and moves into the alert history. One clean incident, opened and closed.

Two honest caveats about the *demo*, which I want to flag up front because they’re orchestration issues, not detection failures:

- One: in attack mode, if you let the script *auto-select* the clearest attack segment, it does that by probing candidate segments — actually calling the scorer. If that scorer is pointed at the live backend, those probe calls can feed the AlertEngine and nudge the alert state before the real run. The workaround is simple — we pass an explicit --attack-segment so it skips probing.
- Two: if you replay *non-adjacent* segments back-to-back, the echoed timestamps can look non-monotonic where the segments join. The detector never uses timestamps — they’re just echoed metadata — so it’s cosmetic. The fix is to replay one contiguous range.

Both are demo-sequencing details. The detection itself is unaffected.”

- **Show:** the dashboard live (backend on :8000, dashboard on :5500), then run `scripts/run_e2e_demo.py --mode attack --attack-segment 7 ...` against the live backend; watch the alert open, then `--mode normal` to close it.
- **Key point:** open → dedup → attribution → close is the full lifecycle; the caveats are orchestration, not ML.
- **Defend if asked:** why the caveats aren’t detection bugs (detector ignores timestamps; probing is a script choice, avoided with an explicit segment).

[PAUSE — “Does that make sense before I get into the numbers?”]

7. Results / Metrics — ~2 min

Say:

“Now the results — and I want to give you the *honest, defensible* numbers, because the honesty here is part of the contribution.

Our **primary, deployment-oriented metrics** are **point-wise**, measured on the clean data at the shipped threshold:

- Precision **0.7822**
- Recall **0.7821**
- F1 **0.7822**
- False-positive rate about **3.01%**

And **threshold-free** — measuring the model’s raw discrimination independent of any cutoff:

- ROC-AUC **0.9429**
- PR-AUC **0.8516**

Now, you may have seen an F1 of **0.9999** in earlier research slides. We deliberately **do not** report that as our result. That number came from a threshold that was *searched on the test set* — the notebook itself warns that column can read near 1.0 even for a broken model. It’s not a deployment number.

There’s also a distinction between **point-wise** and **point-adjusted** metrics. Point-adjusted says: if you catch *any* single point of an attack, you count the *whole* attack segment as detected. It’s common in this literature, but it systematically flatters the score — a near-random detector can look good under it. We report **point-wise** — every window judged on its own — as our primary number, because that’s what actually matters in deployment.

On the ~3% false-positive rate: that’s higher than the 1% the threshold nominally targets, and we can explain exactly why — it’s the calibration caveat from the data issue I mentioned, where six features were frozen on the calibration split. We made a deliberate decision to ship the threshold **unchanged** and **document** the caveat, rather than quietly re-tuning it. Recalibration is a separate, approval-gated experiment we’ve **deferred** — it’s not part of the shipped MVP.

So the story is: strong raw discrimination — ROC-AUC 0.94 — an honest point-wise F1 of 0.78, and a transparent, explained false-positive caveat. No inflated numbers.”

- **Show:** docs/provenance/measured_detection_metrics.md — the “ladder” table showing each protocol change and why the number drops.

- **Key point:** point-wise 0.7822 is the real number; 0.9999 is rejected on principle; the ~3% FPR is explained, not hidden.
- **Emphasize:** the honesty *is* the strength here — a supervisor will respect a defensible 0.78 over an indefensible 0.9999.

[PAUSE — “Does that make sense?”]

8. What the MVP Does NOT Have Yet — ~1.5 min

Say:

“Let me be clear about the boundaries of the MVP — what it deliberately does *not* have yet. None of these are failures; they’re the next engineering stages, and we sequenced them intentionally *after* proving detection works.

- Alerts are **in-memory** — no persistent database yet.
- No **authentication** on the API or dashboard.
- No **Docker** / containerization.
- No **cloud deployment** — no AWS, no Terraform.
- No **CI/CD** pipeline and no **DevSecOps security gates** yet.
- **Kubernetes** is not implemented — we’re treating it as an optional stretch goal.
- And **stuck-channel detection** — gracefully handling a sensor that freezes or drops out — is a hardening item still on the list.

The MVP’s job was to prove the detection system *works* end-to-end. The final project’s job is to prove it’s *deployable, reproducible, monitored, and secured*. These items are that second half.”

- **Show:** README §15 (Known Limitations) — it lists exactly these.
 - **Key point:** these are planned stages, not gaps we overlooked. MVP = “it works”; final = “it’s deployable and secured.”
 - **Avoid:** apologizing for these. State them as roadmap.
-

9. Roadmap: MVP → Final Delivery — ~2 min

Say:

“Here’s the sequence from here to final delivery, and why each step comes in this order.

1. **Hardening / stuck-channel detection** first — before we deploy anything, the system should handle real-world sensor faults gracefully, not just clean replay data. Right now it correctly *rejects* a frozen channel as an operational fault; the hardening is turning that into a graceful, monitored behaviour.
2. **Alert persistence** — move alert state out of memory into a database, so incidents survive a restart. This is the most visible correctness gap, so it’s early.
3. **Docker** — containerize the backend and dashboard. This gives us reproducible, portable deployment and is the prerequisite for everything cloud.
4. **CI/CD** — automate build and test on every change, so the 286 tests run as a gate, not by hand.

5. **DevSecOps security gates** — this is a core theme of the project. On top of CI/CD we add SAST — static analysis — plus dependency scanning and container scanning. The whole point of the platform is *security*, so our own pipeline has to be secured. That's a genuine contribution.
6. **AWS + Terraform** — deploy to the cloud with infrastructure-as-code, so the whole environment is reproducible and version-controlled, not hand-clicked.
7. **Kubernetes** — as an *optional stretch* stage. Because we designed the detector and API to be stateless, they *can* scale horizontally; Kubernetes would demonstrate that, if time allows.
8. **Performance, observability, reliability** — load and latency testing, metrics, logging, tracing.
9. **Final documentation and the final demonstration** — a runbook, the architecture docs, and the full attack demo.

The through-line: the MVP proves '*the detection system works.*' The final project proves '*the detection system is deployable, reproducible, monitored, and secured with DevSecOps practices.*' That's the transition."

- **Show:** README §14 (Roadmap) — implemented vs. future table.
- **Key point:** the ordering is logical — correctness → packaging → automation → security → cloud → scale → polish. Security gates are a *first-class* deliverable, not an afterthought.

[PAUSE — "Does the sequencing make sense to you?"]

10. Closing — ~45 sec

Say:

"To wrap up:

What's complete: a local, end-to-end MVP — SWaT replay, rolling window, a thin stateless API, the TranAD detector, an alert engine with real lifecycle, monitoring endpoints, and a dashboard — with 286 passing tests and a real-HTTP end-to-end run on real data.

What was hard, technically: faithfully reproducing the model's numerics — we lock that with golden vectors — and, honestly, the *data forensics*: finding the duplicated-normal and frozen-feature defects and handling them transparently instead of shipping an inflated number.

What the MVP proves: process-aware attack detection works end-to-end, with honest metrics — ROC-AUC 0.94, point-wise F1 0.78 — and explainable, per-sensor alerts.

What remains: persistence, then Docker, CI/CD, DevSecOps security gates, and cloud deployment.

What the final system will demonstrate: that this detection capability is not just accurate, but *deployable, reproducible, monitored, and secured*. Happy to go deeper on any part."

- **Key point:** confident, honest, clear line between done and planned.
-

Likely Supervisor Questions

Concise spoken answers, all consistent with the repository.

1. Why TranAD? “We compared multiple anomaly-detection models in the research phase and selected it. It gives strong threshold-free discrimination — ROC-AUC 0.94 — on multivariable process data, and because it’s reconstruction-based, we get per-feature error for free, which is our explainability. Selection is closed.”

2. Why SWaT? “It’s a real, labelled, physical water-treatment testbed — the closest public analog to our chlorination use case. Synthetic data wouldn’t be credible for an OT-security claim.”

3. Why 45 features? “That’s the set of process variables the model was trained on — sensors and actuators across the treatment stages. The 45 names and their order are fixed in the model’s artifact contract; we don’t get to choose a different set without retraining.”

4. Why a 30-sample window? “TranAD reasons over a sequence, not a point — an attack shows up as an abnormal *pattern over time*. 30 is the window the model was trained and calibrated on, so it’s part of the fixed contract.”

5. Why 150 seconds of context? “30 samples at an effective 5-second sampling rate is 150 seconds. So every decision is about the last two-and-a-half minutes of plant behaviour, which is enough to capture process dynamics rather than instantaneous noise.”

6. Why not score a single sample? “A single reading has no temporal context — you can’t tell a genuine transient from an attack. Process attacks manifest as abnormal *behaviour over time*, so the model needs the window. The trade-off is that 150 seconds is also our detection-latency floor.”

7. Why is F1 only 0.7822? “Because that’s the *honest* number — point-wise, at the shipped threshold, on clean data. It’s not low; it’s un-inflated. The threshold-free discrimination is ROC-AUC 0.94, which shows the model separates normal from attack well; the F1 reflects operating at a conservative fixed threshold with a documented calibration caveat.”

8. Why is the FPR about 3%? “The threshold nominally targets 1%, but it was calibrated on a split where six of the 45 features were frozen constants due to a data defect. We chose to ship the threshold unchanged and document the caveat rather than quietly re-tune it. Recalibration is a deferred, separate experiment.”

9. Why not report the 0.9999 F1? “Because it’s not real. It came from a threshold searched on the test set — the notebook itself warns that column can read near 1.0 even for a broken model. Reporting it would be dishonest.”

10. What’s point-wise vs. point-adjusted? “Point-wise judges every window independently. Point-adjusted counts an *entire* attack segment as detected if you catch *any* single point in it. Point-adjusted systematically flatters results — a near-random detector scores well under it — so we report point-wise as our primary deployment metric.”

11. Is severity generated by the ML model? “No — and this is important. Severity is a transparent *engineering heuristic*: we band the score-to-threshold ratio — roughly, 2× threshold is MEDIUM, 10× is HIGH. The model emits one score and one threshold; there’s no validated severity classifier, and we label it as a heuristic everywhere.”

12. Why is the detector stateless? “So it’s trivially testable and could scale horizontally later without shared state. All the ‘memory’ the system needs is isolated elsewhere — the buffer in RollingWindow, the alert lifecycle in the AlertEngine. The detector is a pure function: window in, decision out.”

13. Why is the buffer outside FastAPI? “To keep the API thin and stateless. If the backend held per-connection buffers, it couldn’t scale cleanly and would be much harder to test. The caller owns the 30-sample buffer — RollingWindow — and sends complete windows. There’s exactly one place that knows ‘the last 30 samples.’”

14. What happens when the backend restarts? “Right now the alert state is in-memory, so it’s lost on restart. That’s a deliberate MVP boundary. Adding a persistence layer is the next roadmap item, and the alert DTO is already designed to be serializable for exactly that.”

15. Why is alert state in memory? “It was the smallest thing that let us prove the full lifecycle — open, dedup, close — end-to-end. Persistence is real engineering with its own design decisions, so we scoped it as the next stage rather than rushing it into the MVP.”

16. How will Docker help? “Reproducible, portable deployment — the same environment everywhere, no ‘works on my machine.’ It’s also the prerequisite for CI/CD, container scanning, and cloud deployment, so it unlocks the whole second half of the roadmap.”

17. Why do you need DevSecOps? “Because the project is about security — it’d be inconsistent to build a security platform with an insecure pipeline. DevSecOps means security is built into the delivery pipeline: static analysis, dependency scanning, container scanning, all as automated gates. It’s a first-class deliverable, not an add-on.”

18. Why AWS? “To demonstrate real, reproducible cloud deployment with infrastructure-as-code via Terraform — the whole environment version-controlled and rebuildable, rather than manually configured.”

19. Why Kubernetes? “It’s an optional stretch goal. Because we designed the detector and API to be stateless, they can scale horizontally; Kubernetes would demonstrate that orchestration and resilience if time permits. We’re not committing to it as a core deliverable.”

20. What’s the project’s actual contribution beyond TranAD? “TranAD is an existing model — our contribution is the engineering around it: productionizing it into a tested, end-to-end, process-aware monitoring platform; the *data-provenance forensics* that found and documented real dataset defects; honest, defensible metrics instead of inflated ones; and the deployment-and-security path via DevSecOps. The model is one component; the platform and the intellectual honesty are the contribution.”

Presentation Delivery Notes

3 things to emphasize

1. **Intellectual honesty as a strength** — point-wise 0.7822 over an indefensible 0.9999, and a documented FPR caveat. Supervisors respect defensible numbers.
2. **It’s an engineered platform, not a notebook** — 286 tests, real-HTTP end-to-end on real data, clean architectural boundaries.
3. **The data-provenance investigation** — finding the duplicated-

normal and frozen-feature defects is genuine research work, not just plumbing.

3 things NOT to over-explain

1. Transformer / attention internals — “learns normal, flags what it can’t reconstruct” is enough.
2. The exact SWaT row ranges and merge mechanics — summarize the defects, keep the provenance doc in reserve.
3. Framework specifics (CORS, Pydantic validator details) — mention they exist and move on.

3 technical details to be ready to defend

1. **Point-wise vs. point-adjusted** — and *why* point-adjusted overstates performance (a random detector scores well under it).
2. **Why the ~3% FPR is acceptable and expected** — the frozen-feature calibration caveat, and the deliberate decision to ship the threshold unchanged and document it.
3. **Why the boundaries are drawn as they are** — stateless detector, buffer in RollingWindow, single stateful AlertEngine — and how that supports testability and future horizontal scaling.

Status vocabulary (keep it precise)

- **IMPLEMENTED:** SWaT replay, rolling window, stateless detector, thin FastAPI, alert engine (in-memory), monitoring API, dashboard, 286 tests, real-HTTP e2e.
- **DEFERRED:** threshold recalibration (separate, approval-gated experiment — *not* done).
- **PLANNED / FUTURE:** stuck-channel hardening, alert persistence, Docker, CI/CD, DevSecOps gates, AWS + Terraform.
- **STRETCH:** Kubernetes.